FOOSUS

Alimentation Durable & Locale

Contrat de Conception et de Développement de l'Architecture

Projet : Approvisionnement Alimentaire Géographiquement Responsable

Information sur le document

Projet	Approvisionnement Alimentaire Géographiquement Responsable
Client	Foosus
Préparé par	Architecte

Table des matières

Information sur le document 2
Table des matières 3
1. Objet du document 4
2. Introduction et contexte 4
3. La nature de l'accord 5
3.1 Transition entre les deux plateformes - Pattern Strangler Fig 5
4. Objectifs et périmètre 7
4.1 Objectifs business 7
Objectif Business 1 - Scalabilité et résilience 7
Objectif Business 2 - Technologie transparente et expérimentation 7
4.2 Périmètre - Parties prenantes, préoccupations et visions 7
5. Description de l'architecture, principes stratégiques et conditions requises 9
5.1 Principes stratégiques - Architecture Microservices 9
5.2 Principes stratégiques - Communication inter-services 9
5.3 Principes stratégiques - Sécurité (Non Négociables) 9
5.4 Glossaire des termes techniques 10
6. Livrables architecturaux 12
6.1 Développement de l'architecture 12
6.2 Mesures de l'architecture cible - KPIs 12
6.3 Phases de livraison 13
7. Plan de travail commun priorisé 14
7.1 Architecture Decision Records (ADR) 14
7.2 Pipeline CI/CD 14
7.3 Pratiques de Design (UX/UI) 15
7.4 Observabilité 15
8. Plan de communication 17
9. Risques et facteurs de réduction 18
9.1 Structure de gouvernance 18
9.2 Décision critique ouverte - Cloud vs. On-Premise 18
9.3 Analyse des risques 18
10. Hypothèses 20
11. Critères d'acceptation et procédures 21
11.1 Engagements de la Fonction Architecture 21
11.2 Métriques et KPIs de l'État Cible 21
11.3 Procédure d'acceptation 22
12. Procédures de changement de périmètre 23
12.1 Engagements des équipes 23
12.2 Discrétion des équipes - Zones d'autonomie 23
12.3 Processus de résolution des désaccords 24
12.4 Conditions requises pour la conformité 24
13. Calendrier 25

1. Objet du document

Ce contrat définit les standards architecturaux, les pratiques de développement attendues et les engagements mutuels entre la Fonction Architecture et les équipes Design & Développement de Foosus. Il vise à donner de l'autonomie aux équipes tout en garantissant la cohérence nécessaire à la scalabilité de la plateforme.

Plutôt qu'un ensemble de règles imposées de manière descendante, ce contrat est le fruit d'une démarche collaborative. Il repose sur une logique de partenariat, dans laquelle la Fonction Architecture s'engage autant envers les équipes de développement que celles-ci s'engagent envers les principes architecturaux définis.

2. Introduction et contexte

La plateforme actuelle de Foosus a atteint un niveau où elle ne peut plus soutenir les ambitions de croissance et d'expansion de l'entreprise. Après plusieurs années d'évolution, la solution technique est devenue complexe et n'évolue plus suffisamment vite pour suivre le rythme de l'activité, ce qui risque de freiner la croissance.

Les études de marché montrent par ailleurs une tendance claire : les clients souhaitent consommer local et soutenir les producteurs de leur région. Cette niche reste encore peu exploitée par les concurrents. Foosus souhaite donc capitaliser sur les acquis des trois dernières années pour concevoir une nouvelle plateforme capable de connecter efficacement consommateurs, producteurs et artisans locaux, sur l'ensemble des besoins du quotidien.

Les principaux objectifs de l'entreprise sont les suivants :

- Exploiter la géolocalisation pour mettre en relation fournisseurs et consommateurs, avec un calcul de distance intégré.
- Concevoir une architecture évolutive permettant un déploiement progressif sur différentes régions, villes et pays.
- Garantir des déploiements sans interruption de service, sans nécessité de mise hors ligne en production.
- Assurer un accès fluide sur mobile et desktop, en tenant compte des contraintes de bande passante (réseaux cellulaires et haut débit).
- Supporter plusieurs types d'utilisateurs (fournisseurs, consommateurs, back-office), chacun avec des fonctionnalités adaptées.
- Livrer les fonctionnalités de manière incrémentale et régulière afin de permettre une mise en production rapide et itérative.

3. La nature de l'accord

Ce contrat constitue la déclaration d'intention relative à la conception et au développement de l'architecture de la plateforme Foosus – Approvisionnement Alimentaire Géographiquement Responsable (Phase 1). Il formalise les engagements des équipes Design & Développement ainsi que de la Fonction Architecture en matière de livrables, de standards de qualité et de modalités de collaboration.

Le périmètre couvre la Phase 1 du projet, d'une durée de deux trimestres. Ce contrat est amené à être révisé à la fin de chaque trimestre afin de refléter l'évolution du projet. La gouvernance est assurée par l'Architecte d'Entreprise Foosus, sous la supervision de Natasha Jarson (CIO) et avec l'approbation de Ash Callum (PDG).

Les premières analyses des meilleures pratiques architecturales ont mis en évidence plusieurs approches à risque maîtrisé, notamment les microservices, les standards web et mobiles, ainsi que l'utilisation de bases de données éprouvées. Ces orientations constituent le cadre de référence du présent contrat.

3.1 Transition entre les deux plateformes - Pattern Strangler Fig

La coexistence des deux plateformes pendant la transition suit le pattern Strangler Fig : la nouvelle plateforme encapsule progressivement l'ancienne, fonctionnalité par fonctionnalité, jusqu'à remplacement complet.

Mécanisme	Description	Responsable
Feature Flags	Routage conditionnel des utilisateurs vers la nouvelle ou l'ancienne plateforme selon leur profil (early adopters en T1)	Équipes Dev + Pete Parker
API Bridge	La nouvelle plateforme expose une API Gateway permettant d'invoquer les fonctions du système existant pendant la coexistence	Architecte + Tech Lead
Base de données séparée	Aucune base de données partagée entre les deux plateformes - synchronisation via événements si nécessaire	Architecte d'Entreprise
Rollback automatique	Si une fonctionnalité migratoire échoue, le feature flag bascule automatiquement sur l'ancienne plateforme	CI/CD - Jack Harkness
Migration progressive des données	Les données utilisateurs sont migrées au fil des connexions (lazy migration) - pas de migration batch bloquante	Équipes Dev

4. Objectifs et périmètre

4.1 Objectifs business

Objectif Business 1 - Scalabilité et résilience

La pile technologique doit pouvoir évoluer naturellement au rythme de la croissance. L'année dernière, 12 incidents majeurs ont été causés par des changements trop lourds, ce qui met en évidence la nécessité de mieux maîtriser les évolutions du système.

Même en situation de forte charge, la plateforme doit rester accessible, au moins en mode dégradé, grâce à des mécanismes tels que les circuit breakers et les fallback caches. Les limites actuelles de performance ne permettent pas de supporter le niveau d'engagement attendu dans le cadre des futurs programmes marketing.

Objectif Business 2 - Technologie transparente et expérimentation

Les déploiements doivent se faire sans interruption de service, la plateforme étant accessible en continu, 24h/24 et dans plusieurs zones géographiques.

Les équipes produit doivent pouvoir tester différentes variantes, comparer les approches auprès des utilisateurs et mesurer leur impact réel. Elles doivent également disposer d'une bonne visibilité sur l'usage des fonctionnalités, ainsi que de la capacité à revenir en arrière sur certaines décisions architecturales, tant que ces ajustements restent simples et peu coûteux.

4.2 Périmètre - Parties prenantes, préoccupations et visions

Le tableau suivant montre les parties prenantes qui utilisent ce document, leurs préoccupations, et la façon dont le travail d'architecture répondra à ces préoccupations.

Partie prenante	Préoccupation	Vision
Ash Callum (PDG)	Parrain du projet : garantir l'alignement avec la stratégie Foosus et approuver les décisions majeures.	Vision business - croissance vers le prochain million d'utilisateurs, campagnes marketing multi-villes.
Natasha Jarson (CIO)	Superviser la cohérence architecturale, la sécurité et la conformité technique.	Vision système - architecture évolutive, sécurisée, zéro interruption de service.
Daniel Anthony (CPO)	Définir les objectifs produits et les KPI de réussite.	Vision produit - fonctionnalité de recherche géo-consciente, expérimentation, indicateurs de réussite.
Jack Harkness (Resp. Ops)	Évaluation technique et faisabilité opérationnelle.	Vision opérationnelle - réduction incidents P1, déploiements continus, observabilité.
Pete Parker (Resp. Ingénierie)	Coordination des équipes Dev et arbitrage des désaccords techniques.	Vision ingénierie - autonomie des équipes, pipeline CI/CD, couverture tests > 80 %.

Partie prenante	Préoccupation	Vision
Équipes Design / UX	Concevoir des interfaces géo-conscientes performantes sur mobile et desktop.	Vision UX - mobile-first, accessibilité WCAG 2.1 AA, design system partagé, wireframes validés.
Équipes Développement	Implémenter la nouvelle architecture en respectant les standards du contrat.	Vision technique - microservices, ADR, standards de sécurité non négociables.

5. Description de l'architecture, principes stratégiques et conditions requises

5.1 Principes stratégiques - Architecture Microservices

Standard	Règle	Justification
Single Responsibility	Chaque service a une responsabilité métier clairement définie et ne peut pas en posséder une autre.	Isolation des pannes, déploiements indépendants.
Database per Service	Chaque service possède sa propre base de données, inaccessible directement par un autre service.	Découplage des données, indépendance des schémas.
API-First	Toute fonctionnalité est exposée via une API documentée (OpenAPI 3.x) avant toute implémentation d'interface utilisateur.	Contrats stables entre services, testabilité.
Bounded Context	Les modèles de données d'un service restent à l'intérieur de son bounded context - pas de partage de modèles de domaine entre services.	Évite le couplage implicite, DDD-compatible.
Stateless Services	Les services ne maintiennent pas d'état entre les requêtes - l'état est dans la base de données ou le cache distribué.	Auto-scaling horizontal sans session collante.

5.2 Principes stratégiques - Communication inter-services

Cas d'usage	Pattern requis	Exemple concret
Communication synchrone	REST/HTTP via API Gateway	Consommateur → Service Recherche → Service Géolocalisation
Communication asynchrone	Message Queue (event-driven)	Commande passée → Notification e-mail au fournisseur
Circuit breaker	Pattern obligatoire sur les appels inter-services critiques	Service Recherche → Service Géo : fallback sur cache si géo indisponible
Découverte de services	Via API Gateway (routing centralisé)	Pas de hard-coding d'URLs de services internes
Versioning d'API	URI versioning obligatoire (/v1/, /v2/)	Toute breaking change = nouvelle version d'API, ancienne maintenue 2 sprints min.

5.3 Principes stratégiques - Sécurité (Non Négociables)

Les standards de sécurité suivants sont non négociables et vérifiés automatiquement dans le pipeline CI/CD. Tout code ne respectant pas ces standards est rejeté avant merge.

Domaine	Standard requis	Vérification automatique
Authentification	OAuth2/OIDC obligatoire sur tous les endpoints - pas de sécurité maison.	Test automatisé dans CI/CD
Gestion des secrets	Aucun secret, clé API ou mot de passe dans le code ou les fichiers de configuration versionnés.	Scan GitLeaks dans CI/CD
Validation des entrées	Toutes les entrées utilisateurs sont validées et assainies côté serveur sans exception.	Tests DAST en CI/CD
Données personnelles	Les données personnelles (email, localisation) ne doivent jamais apparaître dans les logs.	Analyse statique du code
Dépendances	Toutes les dépendances tierces doivent être à jour et exemptes de CVE critiques.	Scan Snyk/Dependabot dans CI/CD
TLS	TLS 1.3 minimum sur toutes les communications, en interne comme en externe.	Scan automatisé hebdomadaire

5.4 Glossaire des termes techniques

Ce glossaire est destiné aux parties prenantes non techniques. Les équipes de développement peuvent se référer à la documentation complète dans le dépôt GitHub.

Terme	Définition simple
Microservices	Architecture où chaque fonctionnalité (géolocalisation, paiement, recherche...) est un service indépendant, déployable séparément.
ADR (Architecture Decision Record)	Document court qui capture une décision architecturale importante : contexte, options envisagées, décision prise et justification.
API-First	On définit et documente l'interface d'un service avant d'en écrire le code. Permet à plusieurs équipes de travailler en parallèle.
Bounded Context	Frontière claire autour d'un service : ses données et règles métier lui appartiennent exclusivement.
Circuit Breaker	Mécanisme automatique qui arrête les appels vers un service défaillant et bascule sur une solution de repli, évitant les pannes en cascade.
CI/CD	Chaîne automatisée qui teste, vérifie et déploie le code à chaque modification - sans intervention manuelle.
PWA (Progressive Web App)	Application web qui fonctionne comme une app mobile native, y compris en mode offline ou avec une connexion lente.

Terme	Définition simple
SAST/DAST	Analyses automatisées de sécurité du code (statique) et de l'application en fonctionnement (dynamique).
WCAG 2.1 AA	Standard international d'accessibilité web garantissant que les interfaces sont utilisables par les personnes en situation de handicap.
Core Web Vitals	Métriques Google mesurant la performance perçue par l'utilisateur (temps de chargement, réactivité, stabilité visuelle).
Canary Deployment	Technique de déploiement qui expose une nouvelle version à un petit pourcentage d'utilisateurs avant un déploiement complet.
Strangler Fig	Pattern de migration progressive : la nouvelle plateforme remplace l'ancienne fonctionnalité par fonctionnalité, sans rupture de service.

6. Livrables architecturaux

6.1 Développement de l'architecture

La nouvelle architecture Foosus sera construite autour de microservices "géo-conscients", et remplacera progressivement la plateforme existante selon une approche de type *Strangler Fig*, sans interruption de service.

Les premières études architecturales ont permis de définir les orientations suivantes :

- **Microservices à responsabilité unique** : chaque service porte un domaine métier clairement défini et dispose de sa propre base de données.
- **Approche API-first** : toute fonctionnalité est exposée via une API documentée (OpenAPI 3.x) avant toute mise en place d'interface utilisateur.
- **Géolocalisation native** : un service dédié est intégré dès le départ, incluant le calcul de distance, afin de servir de base à de futures innovations liées à la localisation.
- **Support multi-plateforme** : développement d'une PWA *mobile-first*, avec des points de rupture adaptés (320 px, 768 px, 1024 px), optimisée pour les réseaux à faible ou forte bande passante.
- **Communication hybride entre services** : asynchrone via message broker pour les événements non critiques, et synchrone via REST/HTTP et API Gateway pour les appels directs.
- **Déploiement continu** : stratégies blue-green ou canary sans interruption de service, avec rollback automatique en cas d'incident.

La plateforme existante sera maintenue en mode maintenance pendant toute la transition. La migration vers la nouvelle architecture sera pilotée via des feature flags, permettant une montée en charge progressive et contrôlée par les équipes produit.

6.2 Mesures de l'architecture cible - KPIs

Les métriques suivantes sont issues du brief business Foosus (v1.1) et constituent les critères de réussite de l'architecture cible pour la Phase 1. Elles seront mesurées à la fin de chaque trimestre.

Indicateur	Valeur cible	Méthode de mesure	Condition
Adhésions utilisateurs/jour	+10 %	Analytics produit	Score Lighthouse > 85, performance perçue optimisée.
Adhésion producteurs alimentaires	1,4/mois → 4/mois	Dashboard CRM	Fonctionnalité de recherche géo-consciente opérationnelle dès T1.
Délai moyen de parution	3,5 sem. → < 1 sem.	Métriques CI/CD	Pipeline CI/CD automatisé - build, tests, sécurité, déploiement.
Incidents production P1	> 25/mois → < 1/mois	Dashboard ops	Tests > 80 %, SAST, scan dépendances, rollback auto.

Indicateur	Valeur cible	Méthode de mesure	Condition
Disponibilité en surcharge	Service dégradé garanti	Monitoring temps réel	Circuit breaker, fallback cache, architecture résiliente.
Latence géolocalisation	p95 < 500ms mobile	Traces OpenTelemetry	Service géo dédié, CDN, lazy loading, Core Web Vitals CI/CD.

6.3 Phases de livraison

Trimestre	Livrables et jalons clés
T1 - Mois 1	Architecture de base, templates de microservices, pipeline CI/CD initial. Fonctionnalité de recherche géo-consciente (wireframes validés). Décision cloud/on-premise finalisée.
T1 - Mois 2	Intégration géolocalisation & calculateur de distance. Premières livraisons sur la pile technologique cible. Tests de performance initiaux. Feature flags activés pour les early adopters.
T1 - Mois 3	Coexistence des deux plateformes opérationnelle via Strangler Fig. Premiers utilisateurs précoces sur les nouvelles fonctions de recherche. KPIs T1 mesurés et rapportés.
T2 - Mois 4	Tri des offres alimentaires. Gestion multi-utilisateurs (consommateurs, fournisseurs, back-office). Observabilité complète (métriques Prometheus, logs JSON, traces OpenTelemetry).
T2 - Mois 5	Design system partagé stabilisé. Intégrations partenaires identifiées (paiement tiers). Tests de charge en staging avant campagnes marketing.
T2 - Mois 6	Revue générale du contrat. KPIs T2 mesurés. Décision go/no-go Phase 2. Budget Phase 2 présenté au PDG.

7. Plan de travail commun priorisé

7.1 Architecture Decision Records (ADR)

Toute décision architecturale significative doit être documentée dans un ADR avant d'être implémentée. Un ADR est significatif s'il impacte : la structure d'un service, le choix d'une technologie, un pattern de communication, ou un compromis de conception majeur.

Section	Contenu
Titre	ADR-NNN : [Titre court décrivant la décision]
Statut	PROPOSÉ | ACCEPTÉ | DÉPRÉCIÉ | REMPLACÉ PAR ADR-NNN
Contexte	Quel est le problème à résoudre ? Quelles contraintes s'appliquent ?
Décision	Quelle est la décision prise ? (énoncé clair, une phrase)
Options considérées	Quelles alternatives ont été évaluées et pourquoi écartées ?
Conséquences	Quels sont les avantages, inconvénients et risques ?
Impact Business	Comment cette décision sert-elle (ou contraint-elle) les objectifs business ?

7.2 Pipeline CI/CD

Chaque service doit disposer d'un pipeline CI/CD automatisé respectant les étapes suivantes. Aucun code ne peut être déployé en production sans passer toutes ces étapes.

Étape	Activités	Seuil d'échec	Responsable
1. Build	Compilation, vérification syntaxique, linting	Toute erreur bloque	CI automatisé
2. Tests unitaires	Couverture du code métier	< 80 % bloque	Développeurs
3. Tests d'intégration	Tests contrats d'API, interactions inter-services	Tout échec bloque	CI automatisé
4. Sécurité	SAST, scan dépendances, scan de secrets	CVE critique ou secret bloque	CI automatisé
5. Tests de perf.	Tests de charge en staging	Régression > 20 % bloque	CI automatisé
6. Staging	Blue-green deployment en staging	Santé < 5 min	CI automatisé
7. Validation E2E	Tests d'acceptance automatisés sur staging	Tout échec bloque	CI automatisé

Étape	Activités	Seuil d'échec	Responsable
8. Production	Canary/blue-green avec rollback automatique	Erreur > seuil = rollback	CI automatisé

7.3 Pratiques de Design (UX/UI)

Domaine	Standard	Vérification
Mobile-first	Toute interface conçue pour mobile en priorité (breakpoints : 320px, 768px, 1024px)	Tests automatisés + Lighthouse > 85
Performance perçue	Lazy loading, skeleton screens, placeholders pour expérience fluide sur réseaux lents	Core Web Vitals dans CI/CD
PWA	Interfaces développées comme PWA avec support offline basique	Audit Lighthouse PWA > 90
Accessibilité	Conformité WCAG 2.1 niveau AA minimum	Tests accessibilité automatisés dans CI/CD
Design system	Design system partagé créé et maintenu dans le dépôt GitHub	Revue architecturale avant tout ajout
Contraintes géo	Interfaces géo-conscientes respectant les wireframes validés, affichage cartographique optimisé	Revue design obligatoire avant implémentation

7.4 Observabilité

L'observabilité n'est pas optionnelle. Chaque service mis en production doit exposer les trois piliers : métriques, logs et traces.

Pilier	Exigence	Format / Standard
Métriques	Chaque service expose : requêtes/sec, latence p95/p99, taux d'erreur, ressources consommées	Prometheus, endpoint /metrics
Logs	Tous les logs structurés JSON : timestamp, service, level, trace_id, message, contexte	JSON, agrégateur central
Traces	Chaque requête tracée de bout en bout avec propagation du contexte de trace (W3C Trace Context)	OpenTelemetry SDK obligatoire
Health checks	Endpoints /health (liveness) et /ready (readiness) obligatoires, utilisés par l'orchestrateur	JSON avec statut et version

Pilier	Exigence	Format / Standard
Métriques business	Services exposent aussi des métriques métier (conversions, abandons, adhésions) liées aux KPIs	Labels Prometheus avec dimensions métier

8. Plan de communication

Ce plan décrit le rythme, les canaux et les formats utilisés pour la communication entre la Fonction Architecture, les équipes Design & Développement, et les parties prenantes.

Dimension	Contenu
Évènements	Revue d'architecture trimestrielle impliquant l'ensemble des équipes Sessions de design technique à la demande, avec un minimum d'une par sprint et par équipe Revue de code à impact architectural sous 48 heures ouvrées Démonstration des livrables à la fin de chaque sprint pour validation des fonctionnalités et de leur conformité
Canaux	Dépôt GitHub Foosus pour les ADR et la documentation Issues GitHub pour les revues architecturales Sessions de travail collaboratif planifiées E-mail pour les communications formelles entre parties prenantes
Formats	ADR pour toute décision architecturale significative Rapport de sprint Tableau de bord KPI mensuel Contrat révisé trimestriellement
Contenu	Avancement par rapport aux KPI État des livraisons (CI/CD, couverture de tests, incidents P1) Décisions architecturales prises Points de blocage et demandes de dérogation temporaire
Rythme	Hebdomadaire : synchronisation courte équipes / architecte (15 min) Bi-mensuel : revue technique approfondie Trimestriel : revue globale du contrat avec l'ensemble des parties prenantes

9. Risques et facteurs de réduction

9.1 Structure de gouvernance

La gouvernance architecturale est assurée par l'Architecte d'Entreprise Foosus, sous la direction de Natasha Jarson (CIO) et avec le parrainage de Ash Callum (PDG). Pete Parker (Responsable Ingénierie) agit comme point d'escalade opérationnel.

9.2 Décision critique ouverte - Cloud vs. On-Premise

La décision concernant le modèle d'hébergement (cloud ou on-premise) doit être prise avant le démarrage du Sprint 1. En l'absence de cette décision, la conception de l'infrastructure ne peut pas être finalisée.

Les responsables de cette décision sont Natasha Jarson (CIO) et Jack Harkness.

Les critères d'évaluation incluent : coûts opérationnels, latence géographique, conformité RGPD et capacité d'auto-scaling.

Un rapport comparatif fondé sur des données objectives devra être produit au cours du premier mois afin de soutenir la prise de décision.

9.3 Analyse des risques

ID	Risque	Gravité	Prob.	Facteur de réduction	Propriétaire
1	Plateforme existante instable : 12 pannes l'an dernier. Risque de contamination si nouveaux comportements intégrés dans l'ancien système.	Élevée	Élevée	Gel des nouvelles fonctionnalités. Mode maintenance strict. Séparation complète via Strangler Fig.	Pete Parker + Tech Leads
2	Coexistence des deux plateformes : complexité opérationnelle, risque de divergence des données pendant la transition.	Moyenne	Élevée	API-First avec contrats stables. Feature flags. Montée en puissance progressive.	Architecte d'Entreprise
3	Surcharge lors des campagnes marketing : pics non absorbés, causant des pannes.	Élevée	Élevée	Microservices avec auto-scaling horizontal. Circuit breaker. Tests de charge en staging avant chaque campagne.	Jack Harkness (Ops)

ID	Risque	Gravité	Prob.	Facteur de réduction	Propriétaire
4	Sécurité : historiquement, facilité d'utilisation > sécurité, risque réputation.	Élevée	Moyenne	OAuth2/OIDC obligatoire. GitLeaks, SAST, DAST, Snyk dans CI/CD. Standards non négociables.	Architecte + CIO
5	Garant de l'architecture non confirmé : vision architecturale potentiellement non réalisable.	Élevée	Moyenne	Confirmation du garant en priorité. ADR documentés pour préserver la continuité architecturale.	Natasha Jarson (CIO)
6	Décision cloud vs. on-premise non tranchée : débat ouvert sans données suffisantes.	Moyenne	Élevée	Rapport comparatif basé sur données, produit en Mois 1. Décision avant Sprint 1.	CIO + Jack Harkness
7	Autonomie équipes vs. gouvernance : résistance si la gouvernance est perçue comme trop contraignante.	Moyenne	Moyenne	Approche lean. Sessions collaboratives régulières. Sondage trimestriel sur la charge architecturale.	Pete Parker + Architecte
8	Performance géolocalisation sur réseaux lents : expérience dégradée pour utilisateurs mobiles.	Moyenne	Élevée	PWA avec support offline. Lazy loading, skeleton screens. Core Web Vitals mesurés en CI/CD. Latence p95 < 500ms.	Équipes Dev + Design

10. Hypothèses

Le tableau suivant résume les hypothèses retenues pour ce contrat d'architecture.

ID	Hypothèse	Impact	Propriétaire
1	La plateforme existante sera conservée en mode maintenance. Aucune nouvelle fonctionnalité ne sera développée sur celle-ci.	Libère les équipes pour la nouvelle plateforme sans dette supplémentaire.	Ash Callum (PDG)
2	La nouvelle architecture sera construite sur les technologies actuelles avec la capacité de s'adapter à de nouvelles technologies.	Flexibilité long terme ; choix technologiques non imposés.	Architecte d'Entreprise
3	Les équipes étant attachées à la plateforme existante, les dirigeants devront éviter les faux raccourcis intégrant de nouveaux comportements dans l'ancien système.	Risque de contamination architecturale si non respecté.	Pete Parker + Tech Leads
4	Coexistence des deux plateformes durant la transition. Montée en puissance proportionnelle aux fonctionnalités via feature flags.	Pas d'interruption de service ; migration progressive.	Daniel Anthony (CPO)
5	La géolocalisation, si modélisée tôt, permettra d'introduire d'autres innovations basées sur la localisation utilisateur/fournisseur.	Levier de différenciation concurrentielle fort.	Architecte d'Entreprise
6	Une approche architecturale lean sur mesure préservera l'autonomie des équipes et la rapidité des cycles de versions.	Gouvernance explicite sans nuire à l'agilité des équipes.	Architecte + Pete Parker
7	Le budget Phase 1 (T2) sera confirmé avant la fin du Mois 3. En l'absence de confirmation, les livrables T2 seront re-priorisés avec le CPO et la CIO.	Impact direct sur le périmètre de la Phase 1.	Ash Callum (PDG) + Natasha Jarson

11. Critères d'acceptation et procédures

11.1 Engagements de la Fonction Architecture

#	Engagement	Indicateur
1	Toute revue de code architecturale sera fournie dans les 48h ouvrées - jamais bloquante au-delà.	Délai mesuré dans les issues GitHub
2	L'architecte participera aux sessions de design technique (à la demande) pour co-construire les solutions, pas les imposer.	Au moins 1 session collaborative/sprint disponible par équipe
3	Toute contrainte architecturale est justifiée par une raison claire et compréhensible - pas de règle sans explication.	Chaque règle de ce contrat inclut sa justification
4	Un guide de démarrage (Getting Started) pour la nouvelle architecture sera disponible avant le Sprint 1.	Guide publié dans GitHub avant Sprint 1
5	Les templates de microservices seront fournis par la Fonction Architecture pour accélérer le démarrage.	Templates disponibles dans GitHub d'ici fin du Mois 1
6	Le framework d'architecture sera allégé si les équipes remontent qu'il crée un frein à la livraison.	Sondage trimestriel auprès des équipes sur la charge architecturale

11.2 Métriques et KPIs de l'État Cible

De plus, les métriques suivantes seront utilisées pour déterminer le succès de ce travail d'architecture :

Métrique	Technique de mesure	Valeur cible	Justification	Notes
Incidents P1/mois	Dashboard ops (Datadog/PagerDuty)	< 1/mois	Résilience architecture microservices	Baseline actuelle : > 25/mois
Délai de parution	Métriques pipeline CI/CD	< 1 semaine	CI/CD automatisé de bout en bout	Baseline actuelle : 3,5 semaines
Couverture de tests	Rapport couverture CI/CD	> 80 % code métier	Réduction des régressions	Vérification automatique à chaque PR
Score Lighthouse	Audit automatisé CI/CD	> 85 (perf), > 90 (PWA)	Performance perçue et accessibilité	Sur les 3 breakpoints mobiles

Métrique	Technique de mesure	Valeur cible	Justification	Notes
Latence géo p95	Traces OpenTelemetry	< 500ms mobile	Expérience utilisateur sur réseaux lents	Mesurée en conditions réseau simulées 3G

11.3 Procédure d'acceptation

Pour qu'un livrable architectural soit considéré accepté, les conditions suivantes doivent être réunies :

- Tous les KPIs de l'État cible mesurés et conformes aux valeurs cibles définies ci-dessus.
- Revue de code architecturale validée par l'Architecte d'Entreprise (ou dérogation documentée).
- ADR correspondant soumis, revu et au statut ACCEPTÉ dans le dépôt GitHub.
- Pipeline CI/CD passant toutes les étapes sans exception (build, tests, sécurité, performance, staging, E2E, prod).
- Signature du tableau d'approbation en section 14 par toutes les parties.

12. Procédures de changement de périmètre

12.1 Engagements des équipes

#	Engagement	Responsable
A	Toute décision architecturale significative fait l'objet d'un ADR soumis pour revue avant implémentation.	Tech Lead de chaque équipe
B	Les équipes respectent les standards de ce contrat et signalent proactivement toute friction ou impossibilité technique.	Tous les développeurs
C	Aucun service ne communique directement avec la base de données d'un autre service - tout passe par les APIs.	Tech Lead - vérifié en revue de code
D	Les équipes de design impliquent la Fonction Architecture sur les maquettes ayant des implications de performance ou de sécurité significatives.	Lead Designer / UX
E	Les équipes maintiennent une couverture de tests > 80% pour le code de domaine métier.	Tous les développeurs - CI/CD
F	Les équipes participent aux revues d'architecture trimestrielles et remontent leurs retours d'expérience honnêtement.	Pete Parker + Tech Leads
G	Toute dette technique délibérée est documentée dans le backlog technique avec une estimation de son coût d'adressage.	Tech Lead concerné

12.2 Discrétion des équipes - Zones d'autonomie

La Fonction Architecture croit en l'autonomie des équipes. Les choix suivants sont délibérément laissés à leur discrétion, dans le respect des standards définis dans ce contrat.

Domaine de liberté	Options acceptables	Contrainte à respecter
Langage de développement	Node.js, Python, Go, Java, Kotlin... (équipe décide)	API OpenAPI 3.x, standards documentés
Framework applicatif	Express, FastAPI, Spring, NestJS... (équipe décide)	Health checks + métriques Prometheus
Base de données	PostgreSQL, MongoDB, Redis... selon le besoin	Database-per-service, migrations réversibles
Patterns de test	TDD, BDD, testing-library... (équipe décide)	Couverture > 80 % code métier
Organisation interne du code	Architecture hexagonale, clean architecture, layers...	Respect du bounded context, pas de couplage

Domaine de liberté	Options acceptables	Contrainte à respecter
Tooling de développement	IDE, linters, formateurs... (équipe décide)	Linting intégré dans CI/CD

12.3 Processus de résolution des désaccords

En cas de désaccord entre une équipe et la Fonction Architecture sur une décision technique, le processus suivant s'applique :

- Étape 1 : Discussion directe entre le développeur/tech lead et l'architecte pour comprendre les perspectives respectives.
- Étape 2 : Si pas de consensus, escalade à Pete Parker (Responsable Ingénierie) pour arbitrage dans les 3 jours ouvrés.
- Étape 3 : Si l'arbitrage ne satisfait aucune des parties, escalade à la CIO (Natasha Jarson) pour décision finale.
- Étape 4 : La décision et son raisonnement sont documentés dans un ADR, quelle que soit l'issue.

En cas d'urgence de livraison, une dérogation temporaire peut être accordée par l'architecte, à condition que la dette technique correspondante soit documentée et planifiée dans le backlog sous 5 jours.

12.4 Conditions requises pour la conformité

La conformité architecturale est évaluée selon trois niveaux :

Niveau	Définition	Conséquence en cas de non-conformité
Non négociable	Standards de sécurité (§5.3), Database-per-Service, API-First	Blocage immédiat en CI/CD - aucune dérogation possible
Standard fort	Couverture de tests > 80 %, ADR pour décisions significatives, observabilité complète	Dérogation temporaire possible avec documentation dans le backlog sous 5 jours
Recommandation	Choix de langages, frameworks, outils de développement	Libre choix des équipes - aucune dérogation requise

13. Calendrier

Feuille de route Phase 1 - deux trimestres. Budget réservé : Premier trimestre. Budget attendu total : deux trimestres (confirmation avant fin Mois 3).

Période	Livrables et jalons clés	Décisions requises
T1 - Mois 1	Architecture de base, templates microservices, CI/CD initial. Fonctionnalité de recherche géo-consciente (wireframes validés).	Décision cloud/on-premise. Confirmation du garant architecture.
T1 - Mois 2	Géolocalisation & calculateur de distance. Premières livraisons sur la pile cible. Tests de performance initiaux. Feature flags activés.	Validation KPIs T1 intermédiaires.
T1 - Mois 3	Coexistence deux plateformes opérationnelles (Strangler Fig). Premiers early adopters sur nouvelles fonctions. KPIs T1 mesurés.	Confirmation budget T2.
T2 - Mois 4	Tri des offres alimentaires. Multi-utilisateurs. Observabilité complète (métriques, logs, traces).	Validation architecture observabilité par Jack Harkness.
T2 - Mois 5	Design system stabilisé. Intégrations partenaires paiement identifiées. Tests de charge avant campagnes marketing.	Validation plan campagnes marketing (PDG + CPO).
T2 - Mois 6	Revue générale du contrat. KPIs T2 mesurés. Décision go/no-go Phase 2.	Budget Phase 2 présenté au PDG. Signatures go/no-go.